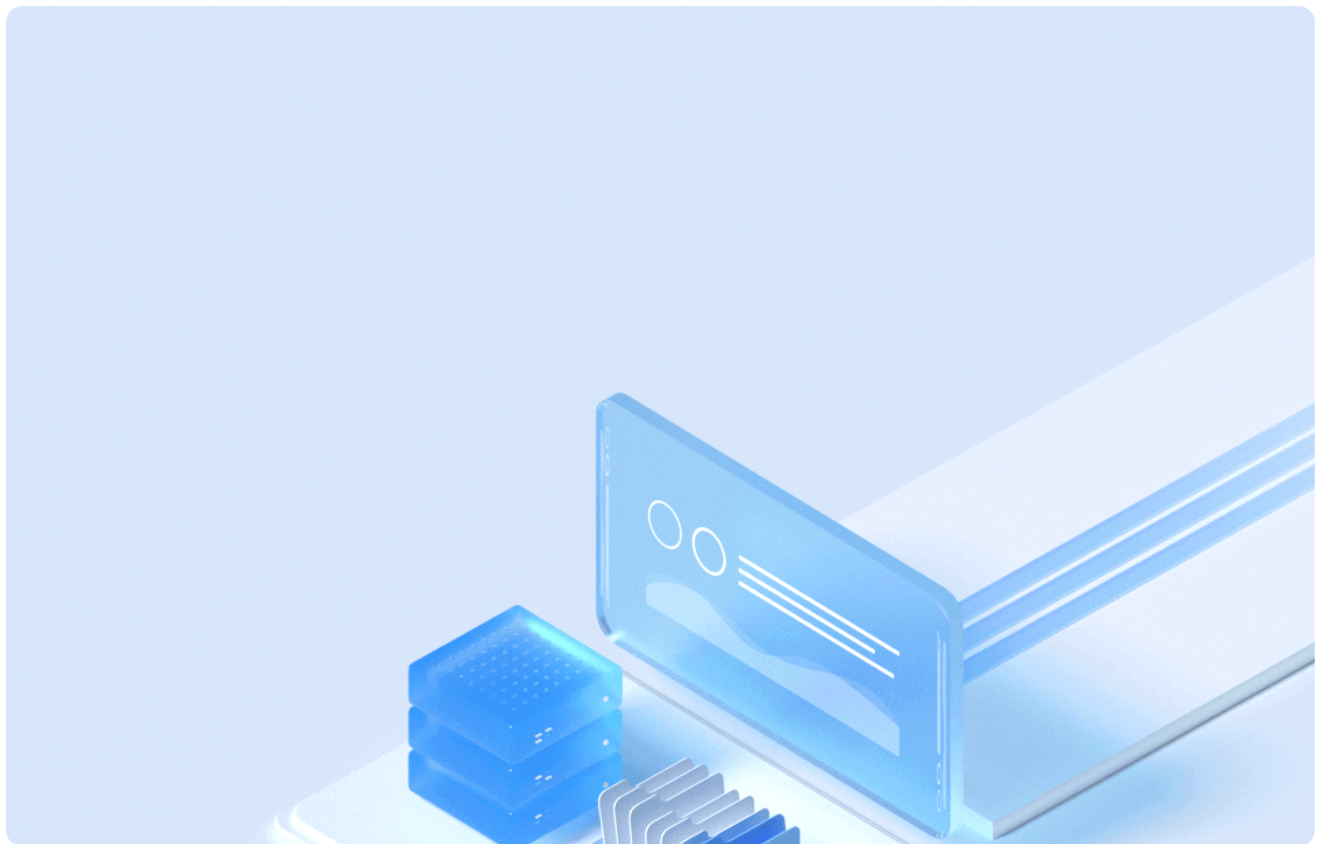


RAGIX

Sovereign document & code intelligence — digest, analyse, compare and present large heterogeneous corpuses, entirely on your own infrastructure.

CAPABILITY TECH SHEET FOR DECISION-MAKERS · VERSION 1.0 · PUBLIC

**PREPARED BY**

Adservio Innovation Lab
Tour Franklin, 100-101 Terrasse Boieldieu, 92800 Puteaux — La
Défense

DATE OF ISSUE

15 June 2026

SCOPE

Sovereign AI for document & code corpuses

AUDIENCE

Executives & engineering leadership

PLATFORM VERSION

RAGIX v0.71 · production-ready

SOURCE

Open source · github.com/ovitrac/RAGIX

Table of contents

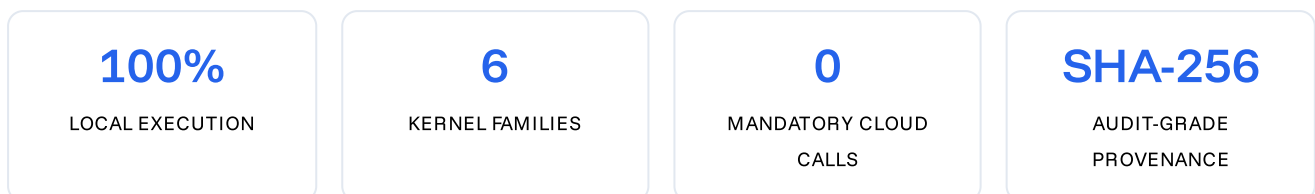
1	RAGIX in one minute — the problem we solve	3
2	What makes RAGIX unique — five differentiators	5
3	The capability catalogue — six kernel families	8
4	Orchestratable by an AI — sovereignly, with proof	11
5	Proof at scale — field case studies	13
6	Operating an advanced engineering forge	16
7	Maturity, openness & technology stack	18

1. RAGIX in one minute

RAGIX (Retrieval-Augmented Generative Interactive eXecution) is a sovereign platform built by Adservio Innovation Lab to make sense of large, messy bodies of knowledge — technical documentation, specifications, audits, regulations and source code — and to turn them into trustworthy deliverables. It reads, structures, summarises, compares, translates and presents corpuses that are far too large for any single person, or any single model, to hold in mind at once. Crucially, it does all of this **on your own infrastructure**: no document ever leaves the building, and every operation leaves an auditable trace.

The platform combines two ingredients that are usually kept apart. Deterministic *kernels* — ordinary, reproducible software — do the measuring, counting and structuring. Local large language models, served privately, are reserved for what they are genuinely good at: planning, interpreting and writing. The numbers are never invented by a model; the prose is never left unchecked. This separation is the foundation of everything that follows.

At a glance, the platform is defined by four numbers.



1.1 The problem we solve

Every large organisation accumulates two kinds of corpus that grow faster than anyone can read them. On one side, **documents**: hundreds of specifications, architecture notes, audit reports and regulatory texts, in mixed formats and languages, written over years by different hands. On the other, **code**: tens of thousands of classes whose behaviour has quietly drifted from what the documents claim. The hardest and most valuable question sits exactly between the two:

THE CORE QUESTION

“Is the code actually doing what the specifications say it does — across thousands of files, millions of lines, and a decade of undocumented change?” Answering this by hand takes weeks of expert time and is never fully reproducible. RAGIX answers it in minutes, with evidence attached to every claim.

For regulated and sovereignty-sensitive sectors — energy, public administration, defence, finance, health — this question carries a second constraint that rules out most off-the-shelf AI tools: the material simply cannot be sent to a third-party cloud. RAGIX was designed from the first line for exactly this setting.

1.2 The RAGIX answer in plain terms

RAGIX behaves like a disciplined team of analysts that never tires and always shows its work. It ingests the corpus, builds a structured map of what is inside, reasons over that map in layers, and produces the deliverable you actually need — an audit report, an executive summary, a comparison, a translation or a finished slide deck. The same engine serves all of these, which is why a single investment covers a wide range of business needs.

Four principles govern every part of the platform and translate directly into business value.

SOVEREIGN

All processing runs inside your perimeter. Air-gap ready, with no dependency on any external API. This is what makes RAGIX usable on regulated, classified or contractually protected material.

LOCAL-FIRST

Reasoning is performed by private local models served through Ollama. No vendor lock-in, predictable cost, and full operation offline — the platform keeps working when the internet does not.

FRUGAL & EXACT

Deterministic kernels compute every metric; models only reason and write. The same input always yields the same numbers, so findings are reproducible and defensible — never a hallucinated figure.

AUDITABLE

Every command, edit and model call is logged in a tamper-evident SHA-256 chain. When a regulator asks how a conclusion was reached, the full provenance is available, down to the source line.

1.3 Who it is for

RAGIX serves three audiences at once. **Decision-makers** receive concise, evidence-backed deliverables and a clear basis for action. **Engineering and audit teams** gain a force multiplier that compresses weeks of manual analysis into hours. **Compliance and risk functions** obtain cryptographic proof that the analysis was performed locally and reproducibly. The remainder of this document explains what sets the platform apart, catalogues its capabilities, demonstrates the sovereign orchestration model, and presents field results from real engagements.

99 The guiding idea, in one line: **make local language models behave like disciplined software engineers — reproducible, auditable and sovereign, with no cloud dependency.**

2. What makes RAGIX unique

Many tools promise AI-assisted analysis. RAGIX is distinguished by five design choices that, taken together, are difficult to find anywhere else — and especially difficult to find in a package that runs entirely on-premises. The table below summarises them; the pages that follow explain each in turn.

★ THE FIVE DIFFERENTIATORS AT A GLANCE		
DIFFERENTIATOR	WHAT IT MEANS	BUSINESS VALUE
Facts & kernels generate documents	Reports are assembled from verified facts and computed values, not free-form prompting	No hallucinated numbers or invented statements; reproducible, defensible deliverables
Sovereign with proof	Runs locally and can prove an external AI never read your content	Usable on regulated and classified material
Layered reasoning	Problems are decomposed, solved and verified in stages	Handles corpuses larger than any context window
Graph-RAG & projections	Retrieval guided by provenance graphs, not similarity alone	Prevents false connections across documents and versions
End-to-end provenance	Every step recorded in a tamper-evident hash chain	Audit-ready evidence on demand

2.1 Facts and kernels generate the document — not prompts

The single most important design decision in RAGIX is the strict division of labour between deterministic kernels and language models. A kernel parses code, counts dependencies, computes a complexity score or builds a graph — identically every time. A language model is asked only to interpret those results, plan the next step, or write the connective sentences. This extends all the way to the **finished document**: a RAGIX report, summary or slide deck is *assembled from verified facts and computed values*, not improvised from a prompt. Every figure, table and metric is machine-derived and carries a pointer to its source; the model writes only the prose that surrounds material that is already true.

This is the practical answer to hallucination, and it is more than a principle — it is how the deliverable is physically built. The numbers cannot drift because the model never authored them; the statements stay anchored because each is tied to an extracted fact. The mechanics of this “facts first, prose last” document generation are detailed in Section 3.5.

✓ WHY IT MATTERS

Re-run the same audit on the same corpus and you obtain the same numbers and the same tables, byte for byte. That reproducibility is what turns an AI-written document into evidence a manager can sign off on and a regulator can accept.

2.2 Sovereign by design — with proof

Running locally is necessary but not sufficient; an organisation also needs to *demonstrate* that sovereignty was respected. RAGIX records a machine-verifiable attestation with every operation — confirming that the work ran on a named host, that the model endpoint was the local service, and that no content left the perimeter. When an external AI assistant is used to orchestrate the work, the platform can prove that the assistant received only metrics and never the documents themselves. Section 4 details this mechanism.

2.3 Reasoning in layers

A specification corpus or a large codebase never fits into a single model prompt. RAGIX therefore reasons in layers: it classifies the problem, decomposes it into sub-problems, solves each independently, then verifies and reconciles the results. Several reasoning engines are available, from a simple plan-and-execute loop to a graph-based state machine and a tree-structured explorer with an adversarial "tutor" that challenges weak answers. The effect is that arbitrarily large material is handled in disciplined, checkable steps rather than one unreliable pass.

2.4 Graph-RAG and data projections

Conventional retrieval finds passages that *look* similar. That is exactly how mistakes are made: a paragraph about a firewall in last quarter's release looks identical to one in this quarter's, and a naive system will merge them and erase the distinction. RAGIX builds a **provenance graph** linking every document, chunk, extracted item and entity, and allows facts to be combined only when they share a genuine neighbourhood in that graph. The same corpus can also be re-projected in several complementary ways for analysis.

The platform offers retrieval in three complementary modes, chosen automatically to fit the task.

UNIX-RAG	PROJECT-RAG	GRAPH-RAG
Live, millisecond exploration of code and history using classic shell tools. Ideal for interactive navigation and ad-hoc questions.	Hybrid keyword-plus-semantic search over an indexed corpus, fusing several ranking strategies for precise concept discovery.	Provenance-constrained synthesis across many documents, keeping versions and sources distinct and every fact traceable.

2.5 Provenance from first read to final word

Trust in an analysis depends on being able to retrace it. In RAGIX, every action — each shell command, each document edit, each model call — is appended to a tamper-evident ledger secured by a SHA-256 hash chain. Each entry references the previous one, so any later alteration is detectable. Outputs carry the hashes of their inputs, which means any finding can be followed back through the analysis to the exact source passage it came from.

This end-to-end provenance underpins three concrete guarantees.

TRACEABILITY

- Every claim links to its source document, page and line
- Findings can be independently re-verified at any time

TAMPER-EVIDENCE

- The hash chain makes silent edits to the record impossible
- Attestations confirm where and how the work was run

Taken together, these five differentiators define a platform that is not merely "AI for documents and code", but a sovereign, evidence-producing instrument. The next section turns from principles to practice and catalogues exactly what RAGIX can do.

THE NET EFFECT

A capability that elsewhere requires sending sensitive material to the cloud, and accepting unverifiable output, becomes — with RAGIX — a fully owned, reproducible and auditable process that never leaves your infrastructure.

3. The capability catalogue

RAGIX organises its work into six families of deterministic kernels. Each family is a coherent business capability — best understood as a verb the platform performs on your corpus. They share a common engine and a common audit trail, so an organisation that adopts one family can extend to the others without changing its tooling or its governance.

# SIX FAMILIES, SIX BUSINESS CAPABILITIES		
FAMILY	IT LETS YOU...	ON MATERIAL SUCH AS...
Audit	Analyse code quality, complexity, coupling and risk	Java / JEE codebases of 10 ³ –10 ⁵ classes
Docs	Digest & summarise hierarchically, with citations	Hundreds of PDFs, Office and Markdown files
Summary	Compare & consolidate across many sources and versions	Multi-vendor, multi-release document sets
Reviewer	Review & edit long documents with full traceability	50+ page specifications and reports
Presenter	Generate slide decks from large reports	Long reports → 25–120 slide decks
Security	Scan for vulnerabilities and compliance gaps	Dependencies, secrets, configuration

Translation and multilingual restitution are not a separate family but a property of the engine: because documents are reduced to structured, citeable units before any prose is generated, the same content can be rendered and summarised in another language — French and English are used routinely in production — with the source meaning preserved and traceable.

3.1 One pipeline, three stages

Every family follows the same disciplined three-stage pipeline. This shared shape is what makes the platform predictable, parallelisable and easy to govern: each stage produces an inspectable result before the next begins.

1. Collection — parse, extract, inventory **2. Analysis** — measure, cluster, score **3. Reporting** — synthesise & render
Collection and analysis are almost entirely deterministic; language models enter only at the reporting stage, and even there their output is checked. Independent kernels within a stage run in parallel, which is why a 60,000-line codebase can be fully analysed in a handful of seconds.

3.2 Understanding code: the Audit family

The Audit family reads a Java or JEE codebase down to its abstract syntax tree and produces a complete, quantified health assessment: complexity hotspots, dead code, architectural coupling, dependency structure, service topology and a risk matrix. It also measures *drift* — the distance between what the documentation claims and what the code actually does — which is the heart of the "does the code match the spec?" question. Results are deterministic and arrive in seconds, not weeks.

3.3 Digesting documents: the Docs and Summary families

The Docs family ingests large mixed-format corpuses, extracts the facts, requirements and definitions they contain, and builds hierarchical, citeable summaries — corpus, domain, cluster and document levels — so a reader can move from a one-page overview to a precise source passage in a few clicks. The Summary family adds Graph-RAG consolidation across many sources and a tiered redaction model, so the same corpus can be released at public, internal or audit-only sensitivity without re-running the analysis.

 **DOCS — DIGEST & SUMMARISE**

- Ingests PDF, Office and Markdown at corpus scale
- Extracts facts, rules and definitions with provenance
- Builds four-level hierarchical summaries
- Detects gaps, overlaps and version drift

 **SUMMARY — COMPARE & CONSOLIDATE**

- Merges sources only along genuine provenance links
- Keeps versions and origins distinct
- Tiered redaction: public / internal / audit
- Visual drift maps between releases

3.4 Producing deliverables: Reviewer & Presenter

The platform does not stop at analysis; it produces the finished artefact. The Reviewer family edits long documents chunk by chunk — fixing errors, flagging inconsistencies — while logging and numbering every change so any edit can be reverted independently. The Presenter family turns a long report into a polished slide deck, at a level of compression you choose, from a full technical deck to a 25-slide executive summary.

 **REVIEWER — REVIEW & EDIT**

- Edits documents beyond any context-window limit
- Every change numbered, reasoned and reversible
- Protected regions (code, maths, tables) untouched
- A fast model proposes; a stronger model validates

 **PRESENTER — GENERATE DECKS**

- Long report → finished slide deck, reproducibly
- Full, compressed or executive compression modes
- Professional theme, diagrams, PDF and HTML output
- Companion handouts and glossaries on demand

3.5 How a RAGIX document is written: facts first, prose last

It is worth being precise about how RAGIX *writes* a deliverable, because it is the opposite of asking a model to "draft a report". The document is built outward from verified material: the corpus is first reduced to structured, citeable facts; the kernels then assemble the document's skeleton — its sections, tables, figures and every number — by computation; and only at the very end does a model add the connective prose around content that is already true. A second model verifies that prose against the facts, and protected regions (figures, tables, code, mathematics) are never rewritten.

Extract citeable facts **Compute** tables & figures **Assemble** the structure **Write** prose around facts **Verify** & lock

This is not a future ambition: the deliverables from Adservio's field engagements were produced this way. Each artefact below is machine-generated, with its numbers and tables computed rather than written, and each statement traceable to a source.

DOCUMENTS RAGIX GENERATES — FACTS IN, DELIVERABLE OUT	
GENERATED ARTEFACT	WHAT IS MACHINE-DERIVED (NOT WRITTEN)
Code audit report (FR/EN)	Every metric, hotspot, coupling table and risk score
Multi-level corpus summary	Fact catalogue, citations, cross-references, appendices
Executive slide deck	Slide structure, figures and quoted findings from the report
Glossary & handouts	Acronyms and definitions extracted from the source corpus

99 THE ONE-LINE MESSAGE

RAGIX does not prompt a model to write your report. It computes the facts, assembles the document, and lets the model phrase what is already verified — so the deliverable is generated, reproducible and evidence-backed by construction.

3.6 Guarding the perimeter: the Security family

The Security family scans dependencies, configuration and source for known vulnerabilities, exposed secrets and compliance gaps, and maps findings to recognised control frameworks. Because it runs locally like every other family, it can be applied to material that could never be uploaded to an external scanning service.

3.7 How you operate it

The same capabilities are reachable through whichever interface suits the user, all backed by the identical kernels and the identical audit trail.

FOR AUTOMATION

Composable command-line tools for each family fit naturally into CI/CD pipelines and scripted, repeatable engagements.

FOR AI ORCHESTRATION

A standard tool protocol exposes the platform to assistants such as Claude, so an AI can run an analysis without ever seeing the content — the subject of the next section.

4. Orchestratable by an AI — sovereignly

A distinctive capability of RAGIX is that a powerful external assistant — such as Claude — can drive an entire analysis without ever reading a single line of your documents. The orchestrator decides *what* to do and reads back only metrics; the content stays sealed inside your perimeter. This reconciles two goals that normally conflict: the convenience of a state-of-the-art AI assistant, and the absolute confidentiality of sovereign material.

The mechanism is a layered "core-and-shell" architecture. Requests travel inward through a guarded gateway; only sanitised results travel back out. The diagram below shows the three layers and what may cross each boundary.

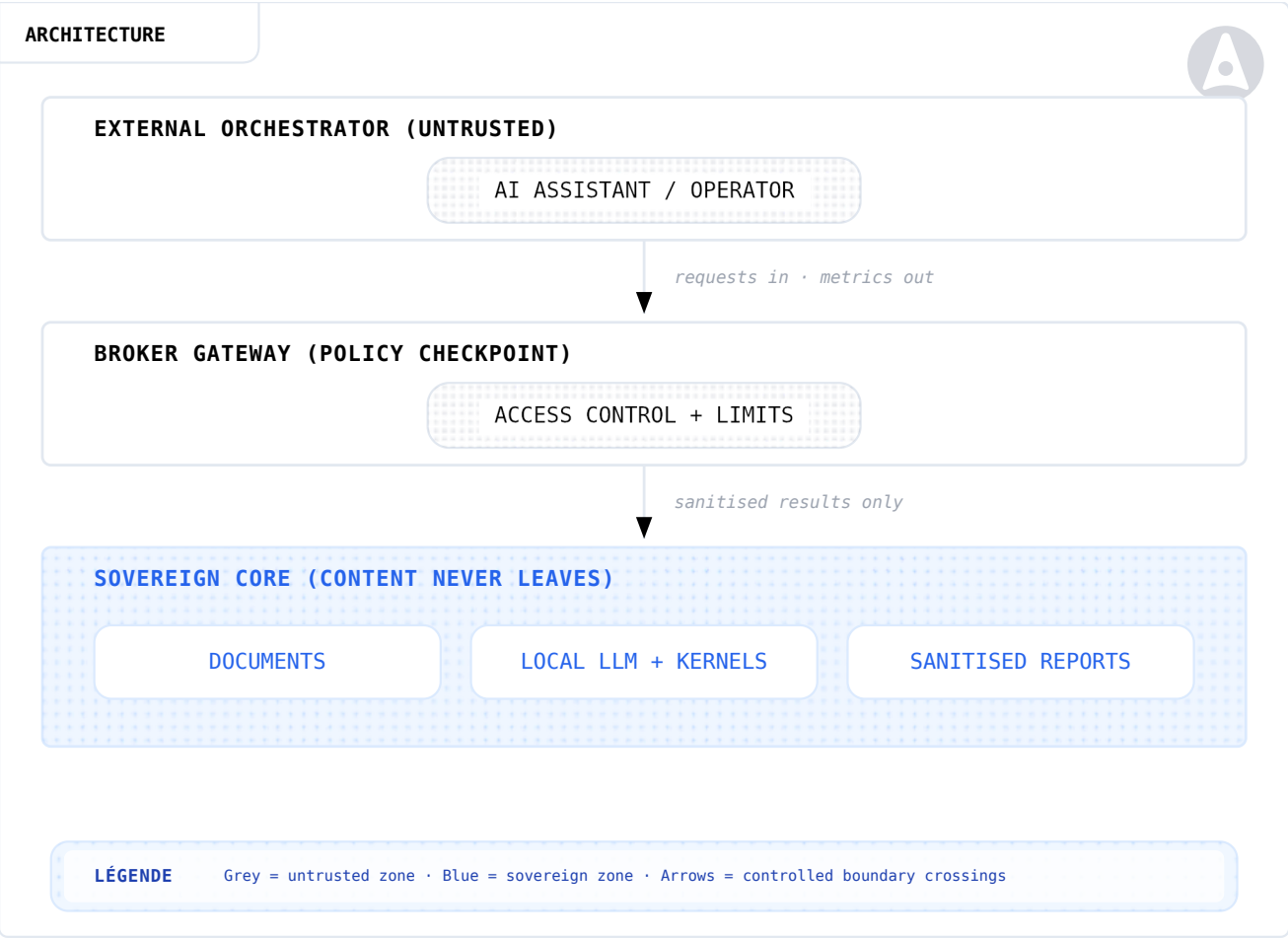


Figure 1 : The core-and-shell architecture — an external AI orchestrates the work but receives metrics, never content.

4.1 Four levels of output isolation

What may cross the boundary is governed by the intended audience. RAGIX enforces this at build time: every kernel result is tagged with an isolation level, and the platform refuses to emit a report in which forbidden internal markers would leak. The four levels make a single analysis safe to share with very different recipients.

 OUTPUT ISOLATION LEVELS			
LEVEL	FOR	CONTAINS	EXCLUDES
Internal	The audit team	Full traces, hashes, paths	Nothing
External	Client delivery	Clean report, metrics, findings	Paths, IDs, internal metadata
Orchestrator	An external AI	Metrics only: counts, timings	All document text
Compliance	A regulator / auditor	Hash chain + attestations	Content (trail only)

4.2 The "proof I never read your documents"

Each operation records a machine-verifiable attestation: that it ran locally, on a named host, against the local model endpoint, with the input and output hashes captured in the audit chain. When an external assistant orchestrates the work under the *orchestrator* isolation level, the record shows it received only metrics. The result is not a policy promise but verifiable evidence.


WHAT YOU CAN TELL A REGULATOR

“The external AI never read the documents. Every model call was local; every boundary crossing carried metrics only; and here is the cryptographic chain that proves it, operation by operation.”

4.3 Why this changes the economics

The combination of a top-tier orchestrating assistant and a sealed sovereign core removes the usual trade-off between capability and confidentiality. Organisations no longer have to choose between the best available AI and the protection of their material; they get both, on their own terms.


THE OLD TRADE-OFF

- Send data to the cloud to use a capable model
- Accept output you cannot verify or reproduce
- Carry the legal and reputational exposure


THE RAGIX POSITION

- Best-in-class assistant orchestrates the work
- Content stays inside the perimeter, always
- Every step reproducible and attested

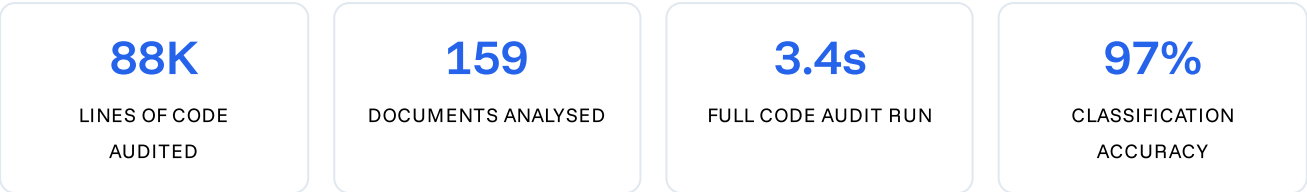
With the principles, capabilities and sovereign operating model established, the natural question from any decision-maker is simply: does it work at scale, on real material? The next section answers with field evidence.


 The same orchestration model lets one operator supervise many analyses at once — and lets Adservio run engagements where the client's data never leaves the client's premises.

5. Proof at scale

RAGIX is not a prototype. It has been used by Adservio on real engagements spanning code audit, document analysis and stakeholder restitution, in sectors where sovereignty is mandatory. The three case studies below are presented in anonymised, sectoral form; the figures are drawn directly from the delivered work.

Across these engagements the platform demonstrated a consistent profile.



5.1 Case A — Enterprise Java platform under maintenance

A mission-critical Java/JEE connectivity platform, maintained by a third party, needed a rapid, objective health assessment for long-term cost and risk planning. RAGIX parsed the whole codebase and delivered a graded report — complete with complexity hotspots, dead-code candidates, coupling analysis and documentation-drift findings — in seconds, in the stakeholders' own language.

CASE A — AT A GLANCE	
DIMENSION	RESULT
Scope	806 source files · 60,157 lines · 582 classes · 2,704 methods
Dependencies mapped	7,396 edges · 27 service components identified
Run time	Full three-stage audit in 3.4 seconds
Outcome	Overall grade A · technical-debt and drift hotspots pinpointed for action

5.2 Case B — Application-separation study, national energy operator

A French national gas-distribution operator needed to know whether two business domains entangled inside one legacy platform could be separated, and at what cost and risk. RAGIX parsed the codebase, built the full dependency graph, and classified each class by domain using local sovereign models. The decisive finding — that cross-domain coupling was extremely low — turned a feared multi-year entanglement into a tractable, evidence-based decision.

DIMENSION	RESULT
Scope	~88,000 lines · 952 classes · 57 modules · 5,834 dependency edges
Key finding	Cross-domain coupling of 0.36% — separation technically viable
Classification	97% of classes auto-classified, validated by client product owners

DIMENSION	RESULT
Outcome	Costed roadmap with two options; ~9% lower five-year cost on the recommended path

The engagement closed with a stakeholder restitution: a concise slide deck and a companion glossary, both generated by the Presenter family from the underlying report, and defended in a working session with the operator's domain owners.

5.3 Case C — Document corpus for a major public administration

A large French municipality held a sprawling corpus of specifications, architecture documents and security assessments accumulated across many contributors and formats. Under a contractual requirement that nothing leave its premises, RAGIX indexed the entire corpus locally, extracted the functional capabilities it described, and surfaced the contradictions and gaps that manual review had missed.

CASE C — AT A GLANCE	
DIMENSION	RESULT
Scope	159 documents (PDF, Office, slides) · 5,515 indexed segments · 9 themes
Extraction	109 distinct functional capabilities catalogued with provenance
Quality findings	51 discrepancies, 33 overlapping pairs and 154 missing references flagged
Sovereignty	100% local — zero external calls, as contractually required


SEE IT IN ACTION

Prefer to watch? A full RAGIX audit demonstration — from raw corpus to finished report — is available on YouTube: youtu.be/vDHI70ZPnDE. It is the quickest way to grasp how the platform turns a large codebase into governed, evidence-backed insight in minutes.

5.4 Multilingual digestion and restitution

Across these engagements RAGIX produced deliverables in both French and English, including full audit reports, executive slide decks, handouts and acronym glossaries for non-specialist audiences. Because the platform reduces a corpus to structured, citeable units before any prose is written, restitution in a second language preserves the source meaning and keeps every statement traceable — a capability of particular value for cross-border and EU-facing work.

5.5 What the field record shows

SPEED THAT CHANGES THE MODEL

Analyses that traditionally take weeks of expert effort complete in seconds to hours — enabling faster, higher-margin engagements without loss of rigour.

SOVEREIGNTY IN PRACTICE

Every engagement ran on regulated or contractually protected material, on-premises, with a complete and reproducible audit trail.

DECISIONS, NOT JUST DATA

Deliverables ended in costed options, prioritised risks and ready-to-present decks — the inputs leaders actually need to act.

REPEATABLE ACROSS DOMAINS

The same engine served code, documents and presentations across energy and public-sector clients — one investment, many uses.

THE TAKEAWAY

RAGIX has already turned weeks-long, hard-to-defend manual analyses into fast, sovereign, evidence-backed deliverables — on exactly the kind of sensitive material where conventional AI tools cannot be used.

6. Operating an advanced engineering forge

A modern **software forge** is the integrated environment in which engineering actually happens: where source code, builds, artefacts, documentation and collaboration live together. As forges grow to hold thousands of repositories and a decade of accumulated knowledge, the hard problem stops being storage and becomes *understanding* — keeping code, specifications and documentation coherent, and keeping that coherence demonstrable to auditors and decision-makers.

This is precisely the layer RAGIX provides. Adservio Innovation Lab positions the platform as the sovereign intelligence that sits over an advanced forge: continuously reading the corpus the forge contains, measuring the distance between intent and implementation, and producing the reports, summaries and presentations that turn raw engineering activity into governed, board-ready insight.

Operating a forge at this level rests on four capabilities, each of which RAGIX already delivers.

GOVERNANCE

Every analysis is reproducible and attested, with tiered outputs for teams, clients, auditors and regulators. The forge becomes not just productive but accountable.

SCALE

Codebases of 10^3 – 10^5 classes and document sets in the hundreds are handled in seconds to hours — the throughput a busy forge requires to stay current.

SOVEREIGNTY

The intelligence layer runs inside the same trusted perimeter as the forge itself — no source, no specification and no secret ever leaves it.

VELOCITY

Audits, consistency checks, summaries and restitutions that once took weeks are produced on demand, letting the forge move faster without losing rigour.

6.1 From repository to decision, continuously

In an Adservio-operated forge, the flow from raw engineering activity to executive decision becomes a continuous, governed loop rather than a periodic, manual effort. Each stage is performed by a RAGIX capability and leaves an audit trail behind it.

Ingest the forge corpus **Analyse** & compare code vs. specs **Generate** reports & decks **Govern** with attested trails
Because an external assistant can orchestrate this loop without reading the content, a single team can supervise the health of an entire forge — surfacing drift, risk and inconsistency the moment they appear, and producing the restitution material leadership needs without a separate documentation effort.

6.2 The Adservio position

Adservio Innovation Lab brings together the two things this mission requires: deep engineering and audit expertise, and a sovereign AI platform built and proven for exactly this work. That combination lets Adservio operate an advanced forge as a governed, intelligent and accountable asset — challenging the assumption that scale, speed and sovereignty cannot be had at once.

 THE AMBITION

To run engineering at scale as a sovereign, fully auditable operation — where the code, the specifications and the story told to leadership are always provably in step, and where the best AI assists without ever compromising confidentiality.

 Everything described here is built on the open-source RAGIX platform and the field experience of Adservio Innovation Lab — not on roadmap promises.

7. Maturity, openness & stack

RAGIX is a working, versioned platform — currently at v0.71, production-ready, and used on the engagements described earlier. It is built entirely on a local-first, open technology stack, which keeps it sovereign, inspectable and free of vendor lock-in. The platform is open source and published on GitHub, so its behaviour can be audited line by line rather than taken on trust.

The major capability families are complete and in production; a focused set of extensions is on the roadmap.

CAPABILITY MATURITY		
CAPABILITY	STATUS	NOTE
Code audit & risk (Audit)	Production	Java / JEE, AST-level, deterministic
Document digestion (Docs)	Production	Hierarchical, citeable summaries
Multi-source synthesis (Summary)	Production	Graph-RAG with tiered redaction
Traceable review (Reviewer)	Production	Reversible, fully logged edits
Slide generation (Presenter)	Production	Full / compressed / executive modes
Security scanning (Security)	Production	Local vulnerability & compliance checks
Additional code languages	Planned	Beyond current Java / Python support
Editable presentation export	Planned	Native office-format decks

7.1 A sovereign, open stack

Each component is chosen so the whole platform can run inside a private perimeter, on commodity hardware, with optional GPU acceleration. There is no required external service.

REASONING & RETRIEVAL

- Local models served via Ollama
- Hybrid keyword + semantic search
- Graph analysis over provenance

ENGINE & OUTPUT

- Python kernels, command-line tools
- Standard tool protocol for AI orchestration
- Reproducible PDF / HTML / slide output

7.2 In summary

RAGIX gives an organisation a single, sovereign instrument for making sense of its documents and its code at scale — and for proving, cryptographically, how every conclusion was reached. It separates exact computation from fluent reasoning, runs entirely on owned infrastructure, can be driven by a world-class AI assistant without ever exposing content, and has been proven on demanding real-world engagements. It is open source, production-ready, and built and operated by Adservio Innovation Lab.



SOVEREIGN & AUDITABLE

Runs on your infrastructure; every step reproducible and attested by a tamper-evident hash chain.



BROAD & COHERENT

One engine to digest, analyse, compare, translate and present — across both documents and code.



PROVEN & FAST

Field-tested at 88K lines and 159 documents; weeks of analysis compressed into seconds and hours.



OPEN & OWNED

Open-source, inspectable, free of vendor lock-in — a platform you can verify and keep.

7.3 Contact

To explore RAGIX, request a live demonstration on your own material, or discuss operating it at the scale of an engineering forge, contact Adservio Innovation Lab.



ADSERVIO INNOVATION LAB

CHANNEL

DETAIL

Contact	Olivier Vitrac, PhD, HDR — Head of Innovation Lab
Email	olivier.vitrac@adservio.fr
Address	Adservio · Tour Franklin, 100-101 Terrasse Boieldieu, 92800 Puteaux — La Défense
Source	github.com/ovitrac/RAGIX (open source)
Demo video	Full audit demonstration — youtu.be/vDHI70ZPnDE

99 Scale, speed and sovereignty — together. That is the challenge RAGIX and Adservio Innovation Lab are built to meet.